Javascript_6

Prototype

JavascriptはPrototype言語である。

Classを使う言語は継承を使うが、JavascriptはPrototypeを使う。

```
> var Person = function (name) {
      this._name = name;
  };

  Person.prototype.getName = function() {
      return this._name;
  };
<· ƒ () {
      return this._name;
  }
> var ppp = new Person('pp');
  ppp.__proto__.getName();
<· undefined
```

Personコンストラクタ関数とgetNameのメソッドをみると、getNameはPrototypeを使うためにPerson.prototype.getNameの形で作る。

そして、pppというインスタンスからPrototypeにアクセスするためにppp.__proto__の形で使用することが見える。

結果を見ればundefinedになる。

メソッドとして呼び出す時にはメソッドの前にあるオブジェクトがthisになる。

ここでthisはpppじゃなくてppp.__proto__になるのでundefinedが出る。

```
> ppp.__proto__._name = 'name_name_ppp';
<· 'name_name_ppp'
> ppp.__proto__.getName();
<· 'name_name_ppp'
```

そして、ppp.__proto__にある_nameがあれば値が出る。

整理すればPrototypeのコンストラクタ関数を作ってオブジェクトを生成すると__proto__と言うPrototypeを参照 するpropertyを使うことができる。

constructor

```
ppp.__proto__.constructor
ƒ (name) {
    this._name = name;
}
```

constructorというpropertyがある。

これはオブジェクトの仕組みを確認する時に使う。

メソッドOverride

```
ppp.getName();
'pp'
```

```
ppp.__proto__.getName();
'name_name_ppp'
```

先ほど見た通り__proto__がある場合とない場合は違う。

```
ppp.getName = function () {
    return 'my name is ' + this.name;
};
ƒ () {
    return 'my name is ' + this.name;
}
```

そして、再定義すれば

```
ppp.getName();
'my name is pp__proto__'
```

my name is pp になるべきだったのにできなかった。

原因をさがすと、

```
ppp.getName = function () {
    return 'my name is ' + this._name;
};
```

```
ƒ () {
    return 'my name is ' + this._name;
}
```

_nameを使ずnameを使ったことが原因だった。

```
ppp.getName();
```

```
'my name is pp'
```

そして、結果的にmy name is ppが出る。